

Содержание

1. Обзор проекта AI Hub Bot	4
1.1 Ключевые принципы	4
1.2 Целевая аудитория	4
2. Архитектура stars-pay-bot: что берём за основу	5
2.1 Технологический стек stars-pay-bot	5
2.2 Механизм 24/7 работы через GitHub Actions	5
2.3 Система платежей Telegram Stars	6
3. Каталог бесплатных AI API	6
3.1 Текстовые AI (чат, генерация, анализ)	6
3.2 Генерация изображений	7
3.3 Аудио и видео	7
3.4 Суммарные возможности бесплатного tier	7
4. Архитектура системы	8
4.1 Общая схема взаимодействия	8
4.2 AI Router - маршрутизация запросов	8
4.3 Rate Limiter - система лимитов	9
4.4 Fallback Chain - отказоустойчивость	9
4.5 Кэширование запросов	9
5. Файловая структура проекта	9

6. Модель данных	10
6.1 Таблица users	11
6.2 Таблица licenses	11
6.3 Таблица ai_usage (лимиты)	11
6.4 Остальные таблицы	12
7. Система тарифов и монетизация	12
7.1 Реферальная программа	13
8. Команды бота и пользовательские сценарии	13
8.1 Основные команды	13
8.2 Inline-режим	14
8.3 Админ-команды	14
9. GitHub Actions: обеспечение 24/7 работы	14
9.1 Основной workflow: run-bot.yml	15
9.2 Keep-alive: keep-alive.yml	15
9.3 Синхронизация данных: sync-data.yml	15
9.4 Секреты GitHub (обязательно)	15
10. Пошаговый план реализации	16
Этап 1: Каркас проекта (1-2 дня)	16
Этап 2: AI интеграция - текст (2-3 дня)	16
Этап 3: AI интеграция - изображения и мультимедиа (2-3 дня)	16
Этап 4: Система платежей и монетизация (1-2 дня)	17

Этап 5: Mini App и публичный API (1-2 дня)	17
Этап 6: Полировка и запуск (1-2 дня)	17
11. Зависимости проекта	17
12. Ключевые фрагменты кода	18
12.1 AI Router - маршрутизация запроса	18
12.2 Обработчик /chat	19
12.3 Платёж Stars - отправка инвойса	19
13. Риски и способы их минимизации	20
14. Конкурентные преимущества проекта	21

1. Обзор проекта AI Hub Bot

AI Hub Bot представляет собой многофункционального Telegram-бота, который объединяет бесплатные AI API для решения широкого спектра задач: генерация текста, создание изображений, обработка видео, транскрипция аудио, перевод и анализ кода. Проект полностью базируется на инфраструктуре GitHub и Telegram, не требуя внешних серверов, баз данных или платных облачных сервисов. Монетизация осуществляется через встроенную платёжную систему Telegram Stars, что позволяет получать оплату напрямую в экосистеме Telegram без подключения платёжных провайдеров.

Архитектура проекта основана на проверенном паттерне из репозитория `sochiautoparts/stars-pay-bot`, который обеспечивает 24/7 работу бота через GitHub Actions с автоперезапуском, хранение данных в SQLite и GitHub Cache, а также синхронизацию лицензий через публичный JSON-файл в репозитории. Ключевое отличие нашего подхода от `sochiautoparts` заключается в том, что все функции работают реально, а не являются заглушками: после оплаты пользователь получает доступ к настоящим AI-сервисам с прозрачной системой лимитов.

1.1 Ключевые принципы

- **GitHub-native:** весь код, данные и автоматизация живут в GitHub репозитории. GitHub Actions обеспечивает 24/7 работу, GitHub Pages - веб-интерфейс, GitHub Cache - персистентность данных.
- **Telegram-native:** бот является единственным интерфейсом пользователя. Telegram Stars - единственная платёжная система. Mini App на GitHub Pages дополняет интерфейс.
- **Multi-AI маршрутизация:** разные задачи направляются к оптимальному AI-провайдеру. Groq для быстрых ответов, OpenRouter для сложных задач, HuggingFace для изображений и специализированных моделей.
- **Freemium модель:** базовые функции бесплатны с ограничениями, расширенные возможности доступны через подписку за Telegram Stars. Лимиты реальны и контролируются серверно.
- **Fallback цепочки:** если основной AI-провайдер недоступен или исчерпан лимит, запрос автоматически перенаправляется к следующему провайдеру в цепочке. Это обеспечивает отказоустойчивость без участия пользователя.

1.2 Целевая аудитория

Проект ориентирован на широкую аудиторию пользователей Telegram, которым нужен удобный доступ к AI-инструментам без необходимости регистрироваться на

множестве сервисов, разбираться в API-ключах и платить за подписки каждому провайдеру отдельно. Основные сегменты: студенты и школьники, использующие AI для учёбы; контент-мейкеры, которым нужны тексты и изображения; разработчики, ищущие быстрые ответы на технические вопросы; предприниматели, использующие AI для бизнес-задач. Бот решает проблему фрагментации AI-сервисов, предоставляя единый интерфейс ко всем инструментам.

2. Архитектура stars-pay-bot: что берём за основу

Репозиторий `sochiautoparts/stars-pay-bot` является универсальной платформой оплаты через Telegram Stars, которую мы используем как базовый шаблон. Ниже представлен детальный анализ его архитектуры с указанием компонентов, которые мы переносим, модифицируем или заменяем.

2.1 Технологический стек stars-pay-bot

Компонент	Технология	Назначение
Telegram Bot	aiogram 3.x	Обработка команд, платежей, callback-запросов
База данных	SQLite +aiosqlite	Пользователи, лицензии, платежи, рефералы
REST API	Flask	Проверка лицензий внешними проектами
Хостинг 24/7	GitHub Actions (cron)	Бот запускается каждые 5 часов с автоперезапуском
Keep-alive	GitHub Actions (cron)	Проверка каждые 30 мин, перезапуск при падении
Mini App	GitHub Pages	Веб-интерфейс оплаты в Telegram
Лицензии	JSON в репозитории	Публичная проверка без сервера через <code>raw.githubusercontent.com</code>
Персистентность	GitHub Cache	SQLite БД и <code>licenses.json</code> сохраняются между запусками
Платежи	Telegram Stars (XTR)	Встроенная платёжная система Telegram

Таблица 1. Технологический стек stars-pay-bot

2.2 Механизм 24/7 работы через GitHub Actions

Ключевая инновация stars-pay-bot - обеспечение круглосуточной работы бота без внешнего сервера, исключительно за счёт бесплатных возможностей GitHub Actions для публичных репозиторийев. Механизм работает следующим образом: основной

workflow (run-bot.yml) запускается по cron каждые 5 часов и работает до 345 минут (5ч45м), после чего автоматически завершается. Параллельно второй workflow(keep-alive.yml) проверяет каждые 30 минут, активен ли основной бот, и при необходимости инициирует перезапуск через GitHub API. Это обеспечивает практически непрерывную работу с минимальными окнами простоя между сессиями.

После каждой сессии бот синхронизирует данные: SQLite база данных сохраняется в GitHub Cache (actions/cache), а лицензии экспортируются в файл data/licenses.json, который коммитится прямо в репозиторий. Этот файл доступен по публичному URL raw.githubusercontent.com, что позволяет другим проектам проверять лицензии без обращения к серверу или API. Таким образом, даже если бот временно недоступен, проверка лицензий продолжает работать.

2.3 Система платежей Telegram Stars

Telegram Stars (валюта XTR) - это встроенная платёжная система Telegram, позволяющая принимать оплату за цифровые товары и услуги напрямую в мессенджере. Для отправки инвойса бот вызывает метод send_invoice с currency="XTR" и provider_token="" (пустая строка для Stars). После подтверждения предчекаута (pre_checkout_query) и успешной оплаты бот получает событие successful_payment с уникальным идентификатором платежа. Вся обработка происходит в рамках Telegram, без интеграции с внешними платёжными системами, что критически важно для нашей GitHub-native архитектуры.

3. Каталог бесплатных AI API

Ниже представлен полный каталог бесплатных AI API, которые могут быть подключены к проекту. Для каждого провайдера указаны лимиты, доступные модели и оптимальные сценарии использования. Ключевая стратегия - использовать разные провайдеры для разных задач, максимизируя суммарный бесплатный лимит.

3.1 Текстовые AI (чат, генерация, анализ)

Провайдер	Модели	Бесплатный лимит	Ключ
Groq	Llama 3.3 70B, Mixtral 8x7B, Gemma 2 9B	14 400 запросов/день, сверхбыстрый (800+ токенов/сек)	groq.com API key

OpenRouter	DeepSeek V4 Flash, Llama 3.1 8B, Mistral 7B (27+ бесплатных моделей)	Без жёсткого лимита, rate limit по IP	openrouter.ai API key
GitHub Models	GPT-4o mini, Llama 3.1 70B, Mistral Large, Phi-3	~200 запросов/день, 60/мин	GitHub PAT token
Google Gemini	Gemini 2.0 Flash, Gemini 1.5 Flash	1 500 запросов/день, 15/мин	Google AI API key
HuggingFace	500+ моделей через Serverless Inference API	Ограничено по ресурсам, per-model rate limit	HF Access Token

Таблица 2. Текстовые AI провайдеры

3.2 Генерация изображений

Провайдер	Модель	Лимит	Размер
Pollinations.ai	Flux (auto)	Безлимитно, без API key	До 1024x1024
HuggingFace	Stable Diffusion XL, FLUX.1-schnell	Serverless, rate-limited	До 1024x1024
Prodia	SDXL, Deliberate, DreamShaper	Бесплатный tier	До 1024x1024

Таблица 3. Провайдеры генерации изображений

3.3 Аудио и видео

Провайдер	Функция	Модель/Инструмент	Лимит
HuggingFace	Speech-to-Text	Whisper Large V3	Serverless, rate-limited
HuggingFace	Text-to-Speech	Bark, SpeechT5	Serverless, rate-limited
HuggingFace	Перевод	NLLB-200, SeamlessM4T	Serverless, rate-limited
Pollinations	Генерация видео	T2V модели (ограничено)	Экспериментальный

Таблица 4. Провайдеры аудио и видео

3.4 Суммарные возможности бесплатного tier

При комбинировании всех бесплатных AI API совокупный лимит составляет примерно 16 000+ текстовых запросов в день, неограниченное количество генераций изображений через Pollinations, а также доступ к транскрипции, синтезу речи и переводу через HuggingFace. Этого более чем достаточно для обслуживания сотен бесплатных пользователей и десятков платных подписчиков. Ключевой принцип маршрутизации: сначала используем провайдера с наибольшим лимитом (Groq), при исчерпании переключаемся на следующий в цепочке (OpenRouter, затем GitHub Models, затем Gemini). Для генерации изображений Pollinations используется как основной провайдер благодаря отсутствию лимитов и требований к API ключу.

4. Архитектура системы

4.1 Общая схема взаимодействия

Система состоит из четырёх основных слоёв, каждый из которых выполняет чётко определённую роль. Слой пользователя (Telegram) обеспечивает интерфейс взаимодействия через команды бота, inline-режим и Mini App. Слой маршрутизации (AI Router) определяет тип запроса, выбирает оптимального AI-провайдера, управляет лимитами и обрабатывает fallback-цепочки. Слой интеграции (AI Providers) содержит адаптеры для каждого подключённого AI-сервиса с унифицированным интерфейсом. Слой хранения (Storage) обеспечивает персистентность данных через SQLite, GitHub Cache и JSON-файлы в репозитории.

4.2 AI Router - маршрутизация запросов

AI Router является центральным компонентом системы, принимающим решение о том, какой AI-провайдер обработает конкретный запрос. Логика маршрутизации учитывает три фактора: тип задачи (текст, изображение, аудио, код), текущие лимиты провайдеров и приоритетную цепочку для каждого типа задач. При поступлении запроса Router определяет его категорию, проверяет доступность и остаток лимитов у приоритетного провайдера, и при необходимости перенаправляет к следующему в цепочке.

Приоритетные цепочки маршрутизации:

- **Текст (чат):** Groq (быстрый, 14 400/день) -> OpenRouter (27+ бесплатных моделей) -> GitHub Models (200/день) -> Gemini (1 500/день) -> HuggingFace (резерв)
- **Генерация изображений:** Pollinations (безлимит, без ключа) -> HuggingFace SDXL -> Prodia
- **Транскрипция аудио:** HuggingFace Whisper -> Groq Whisper API
- **Перевод:** HuggingFace NLLB-200 -> Groq (через промпт) -> Gemini

- **Анализ кода:** Groq (Llama 3.3 70B) -> OpenRouter (DeepSeek V4) -> GitHub Models
- **Генерация видео:** Pollinations (экспериментальный) -> HuggingFace

4.3 Rate Limiter - система лимитов

Rate Limiter отслеживает использование каждого AI-провайдера и обеспечивает соблюдение бесплатных лимитов. Лимиты хранятся в SQLite с полями provider, model, date, count, last_reset. При каждом запросе система проверяет, не исчерпан ли дневной лимит провайдера, и при необходимости переключает на следующий в fallback-цепочке. Для платных пользователей лимиты увеличены: Free tier получает базовый доступ (10 текстовых запросов/день, 2 изображения/день), Pro tier - расширенный (200 текстовых запросов/день, 20 изображений/день), Ultra tier - максимальный (безлимит в рамках возможностей API провайдеров).

4.4 Fallback Chain - отказоустойчивость

Fallback Chain обеспечивает бесперебойную работу даже при недоступности отдельных провайдеров. Если основной провайдер возвращает ошибку (превышен лимит, сервис недоступен, таймаут), система автоматически перенаправляет запрос к следующему провайдеру в приоритетной цепочке. Каждый провайдер в цепочке имеет timeout (по умолчанию 10 секунд для текста, 30 секунд для изображений). Если все провайдеры в цепочке недоступны, пользователь получает вежливое сообщение с просьбой повторить попытку позже. Все ошибки логируются для последующего анализа и оптимизации цепочек.

4.5 Кэширование запросов

Для оптимизации использования AI API и ускорения ответов реализована двухуровневая система кэширования. Первый уровень - in-memory кэш (LRU, до 1000 записей), хранящий результаты недавних идентичных запросов. Второй уровень - SQLite-кэш, сохраняющий пары "запрос-ответ" с хэшем запроса в качестве ключа. Кэш действует 24 часа для текстовых запросов и 7 дней для изображений. Это позволяет мгновенно возвращать ответ на повторный запрос без обращения к AI API, что значительно экономит лимиты провайдеров.

5. Файловая структура проекта

Проект следует модульной архитектуре, где каждый компонент выделен в отдельный пакет Python. Структура файла организована для максимальной понятности и простоты развития проекта.

```
ai-hub-bot/  
  bot/ # Telegram Bot (aiogram 3.x)
```

```

__init__.py
main.py # Точка входа, запуск бота и API сервера
config.py # Конфигурация (env vars, продукты, тарифы)
database.py # SQLite: пользователи, лицензии, платежи, лимиты
handlers/ # Обработчики команд и callback-запросов
    __init__.py
    start.py # /start, реферальная система
    chat.py # Обработка текстовых сообщений -> AI Router
    image.py # /image, генерация изображений
    audio.py # /transcribe, /tts, транскрипция и синтез
    translate.py # /translate, перевод текста
    code.py # /code, анализ и генерация кода
    payment.py # Stars платежи, инвойсы, лицензии
    admin.py # Админ-команды: статистика, управление
middleware.py # Обработка ошибок, логирование
keyboards.py # Inline-клавиатуры (меню, тарифы)
ai/ # AI интеграции
    __init__.py
    router.py # AI Router: маршрутизация запросов
    rate_limiter.py # Отслеживание лимитов провайдеров
    cache.py # Двухуровневое кэширование (memory + SQLite)
    providers/ # Адаптеры AI провайдеров
        __init__.py
        base.py # Базовый класс провайдера
        groq_provider.py # Groq API адаптер
        openrouter_provider.py # OpenRouter API адаптер
        github_provider.py # GitHub Models адаптер
        gemini_provider.py # Google Gemini адаптер
        huggingface_provider.py # HuggingFace Serverless адаптер
        pollinations_provider.py # Pollinations.ai адаптер (изображения)
        prodia_provider.py # Prodia API адаптер (изображения)
api/ # REST API (Flask)
    __init__.py
    server.py # Сервер проверки лицензий
data/ # Публичные данные (коммитятся в репо)
    licenses.json # Публичный API лицензий
    stats.json # Публичная статистика бота
miniapp/ # Mini App (GitHub Pages)
    index.html
    css/style.css
    js/app.js
.github/workflows/
    run-bot.yml # 24/7 бот (каждые 5 часов)
    keep-alive.yml # Проверка каждые 30 мин
    deploy.yml # Deploy Mini App на GitHub Pages
    sync-data.yml # Синхронизация лицензий и статистики
requirements.txt # Зависимости Python
Dockerfile # Docker (опционально, для Render)
docker-compose.yml # Docker Compose (опционально)
render.yaml # Render.com деплой (опционально)
README.md # Документация проекта

```

6. Модель данных

База данных SQLite содержит 7 основных таблиц, обеспечивающих все функции бота: от управления пользователями до отслеживания AI-лимитов. Схема спроектирована с учётом расширяемости - при добавлении новых AI-провайдеров или

функций достаточно добавить записи в существующие таблицы.

6.1 Таблица users

Поле	Тип	Описание
user_id	INTEGER PK	Telegram ID пользователя
username	TEXT	Telegram username
first_name	TEXT	Имя пользователя
language_code	TEXT	Код языка (ru, en и т.д.)
referred_by	INTEGER	ID реферера
referral_code	TEXT UNIQUE	Уникальный реферальный код
subscription_tier	TEXT	Уровень подписки: free / pro / ultra
created_at	REAL	Unix timestamp регистрации

Таблица 5. Структура таблицы users

6.2 Таблица licenses

Поле	Тип	Описание
key	TEXT PK	Лицензионный ключ (AIP-PRO-A1B2-C3D4)
user_id	INTEGER FK	Владелец лицензии
project	TEXT	ai-hub-bot (единый проект)
plan	TEXT	free / pro / ultra
activated_at	REAL	Дата активации
expires_at	REAL	Дата истечения (0 = бессрочно)
active	INTEGER	1 = активна, 0 = деактивирована

Таблица 6. Структура таблицы licenses

6.3 Таблица ai_usage (лимиты)

Поле	Тип	Описание
------	-----	----------

user_id	INTEGER	Telegram ID пользователя
task_type	TEXT	Тип задачи: text / image / audio / code / translate
provider	TEXT	Использованный AI провайдер
date	TEXT	Дата в формате YYYY-MM-DD
count	INTEGER	Количество запросов за день
tokens_used	INTEGER	Общее количество токенов

Таблица 7. Структура таблицы ai_usage

6.4 Остальные таблицы

Таблица **payments** хранит историю платежей (user_id, project, plan, stars_amount, telegram_charge_id, provider_charge_id, license_key, created_at). Таблица **referrals** отслеживает реферальные связи (referrer_id, referred_id, bonus_given, created_at). Таблица **api_keys** управляет API-ключами для внешнего доступа (key, project, description, created_at). Таблица **provider_limits** хранит дневные лимиты AI-провайдеров (provider, model, daily_limit, used_today, last_reset). Все таблицы используют индексы для быстрого поиска по user_id, project и date.

7. Система тарифов и монетизация

Монетизация построена по модели freemium с тремя уровнями подписки. Бесплатный tier привлекает пользователей и демонстрирует ценность продукта, платные тарифы предоставляют расширенные возможности. Оплата осуществляется через Telegram Stars, что обеспечивает мгновенную обработку платежей без комиссий для пользователей и без необходимости интеграции с платёжными системами.

Параметр	Free	Pro	Ultra
Цена	0 Stars	149 Stars/мес	499 Stars/мес
Годовая подписка	-	999 Stars/год	3 499 Stars/год
Навсегда	-	2 999 Stars	9 999 Stars
Текстовые запросы/день	10	200	Безлимит*
Генерация картинок/день	3	30	100

Транскрипция аудио/день	1	20	50
Перевод/день	5	100	Безлимит
Контекст (токены)	4 000	32 000	128 000
Доступ к быстрым моделям	Нет	Да	Да + приоритет
История чата	Не сохраняется	7 дней	30 дней
Реферальный бонус	10 Stars	25 Stars	50 Stars

Таблица 8. Сравнение тарифных планов

* Безлимитные текстовые запросы для Ultra tier означают отсутствие искусственного ограничения со стороны бота. Фактический лимит определяется возможностями подключённых AI-провайдеров (16 000+ запросов/день суммарно от всех бесплатных API). Это значительно превышает потребности любого отдельного пользователя.

7.1 Реферальная программа

Реферальная программа является ключевым инструментом вирусного роста. Каждый пользователь получает уникальную реферальную ссылку вида https://t.me/ai_hub_bot?start=REFCODE. Когда новый пользователь переходит по ссылке и оплачивает любой тариф, реферер получает бонус в Telegram Stars (сумма зависит от его тарифа). Бонусы накапливаются на балансе и могут быть использованы для продления подписки. Механизм реализован через таблицу referrals в SQLite и обрабатывается автоматически при успешной оплате.

8. Команды бота и пользовательские сценарии

8.1 Основные команды

Команда	Описание	Tier
/start [ref]	Запуск бота, обработка реферальной ссылки	Free
/chat [текст]	Чат с AI, отправка текстового вопроса	Free
/image [запрос]	Генерация изображения по описанию	Free
/transcribe	Транскрипция голосового/аудио сообщения	Free
/tts [текст]	Синтез речи из текста	Pro+
/translate [текст]	Автоматический перевод текста	Free

/code [вопрос]	Помощь с кодом, анализ, генерация	Free
/models	Список доступных AI моделей	Free
/limits	Текущие лимиты и использование	Free
/subscribe	Выбор тарифа и оплата Stars	Free
/mylicenses	Просмотр активных лицензий	Free
/refer	Реферальная ссылка и статистика	Free
/settings	Настройки: язык, модель по умолчанию	Free

Таблица 9. Команды бота

8.2 Inline-режим

Бот поддерживает inline-режим, позволяя пользователям обращаться к AI прямо из любого чата Telegram, набирая `@ai_hub_bot` и запрос. Это значительно расширяет охват: пользователи могут использовать AI в рабочих чатах, группах и каналах без перехода в диалог с ботом. Результаты inline-запросов кэшируются на 5 минут для ускорения повторных обращений. Inline-режим доступен только для текстовых запросов (чат, перевод, код) и требует активной подписки Pro или Ultra для полноценной работы.

8.3 Админ-команды

Команда	Описание
/admin	Статистика: пользователи, платежи, использование AI, лимиты провайдеров
/addplan tier price days	Добавление или изменение тарифного плана
/genkey tier user_id	Ручная генерация лицензионного ключа
/resetlimits user_id	Сброс дневных лимитов пользователя
/providerstats	Статистика использования AI провайдеров
/broadcast текст	Рассылка сообщения всем пользователям

Таблица 10. Админ-команды

9. GitHub Actions: обеспечение 24/7 работы

Бот работает 24/7 исключительно за счёт бесплатных GitHub Actions для публичных репозиторий. Система состоит из четырёх workflow, каждый из которых выполняет свою задачу. Важно понимать, что GitHub Actions для публичных репозиторий не имеет ограничений по минутам, что делает эту схему полностью бесплатной.

9.1 Основной workflow: run-bot.yml

Запускается по cron каждые 5 часов (0 */5 * * *) и по нажатию кнопки. Каждая сессия работает до 345 минут (5ч 45м), после чего бот корректно завершает работу. Перекрытие сессий (~15 минут) гарантирует отсутствие простоев. При каждом запуске workflow восстанавливает SQLite базу из GitHub Cache, устанавливает зависимости и запускает бота командой `python -m bot.main`. После завершения сессии данные сохраняются обратно в кэш, а `licenses.json` и `stats.json` коммитятся в репозиторий.

9.2 Keep-alive: keep-alive.yml

Запускается каждые 30 минут (* /30 * * * *) и проверяет наличие активного запуска основного workflow через GitHub API. Если активных запусков нет, автоматически инициирует новый запуск run-bot.yml. Требуется наличие GitHub PAT token (GH_PAT_TOKEN) с правами actions:write. Это гарантирует, что бот не будет простаивать более 30 минут даже при непредвиденных сбоях.

9.3 Синхронизация данных: sync-data.yml

Запускается каждый час и выполняет три задачи: экспорт лицензий из SQLite в data/licenses.json (публичный API), экспорт статистики в data/stats.json (публичная статистика для README), коммит изменений в репозиторий. Это обеспечивает доступность данных о лицензиях даже когда бот временно не работает.

9.4 Секреты GitHub (обязательно)

Секрет	Значение	Описание
BOT_TOKEN	Токен от @BotFather	Токен Telegram бота
ADMIN_IDS	123456789	Telegram ID администраторов через запятую
GROQ_API_KEY	gsk_...	API ключ Groq (бесплатный на groq.com)
OPENROUTER_KEY	sk-or-...	API ключ OpenRouter (бесплатный)

GITHUB_TOKEN	ghp_...	GitHub PAT для GitHub Models
GEMINI_KEY	AIza...	Google AI API ключ
HF_TOKEN	hf_...	HuggingFace Access Token
GH_PAT_TOKEN	ghp_...	GitHub PAT для keep-alive (actions:write)

Таблица 11. Секреты GitHub

10. Пошаговый план реализации

Реализация проекта разбита на 6 этапов, каждый из которых может быть завершён за 1-3 дня. Общий срок реализации - 10-14 дней при работе одного разработчика. Каждый этап produces рабочий инкремент, который можно тестировать и деплоить.

Этап 1: Каркас проекта (1-2 дня)

- Создать GitHub репозиторий ai-hub-bot, настроить структуру директорий
- Реализовать bot/main.py, bot/config.py, bot/database.py на основе stars-pay-bot
- Настроить /start, /help, базовые клавиатуры
- Настроить GitHub Actions: run-bot.yml и keep-alive.yml
- Проверить: бот запускается, отвечает на /start, работает 24/7

Этап 2: AI интеграция - текст (2-3 дня)

- Реализовать ai/router.py с приоритетными цепочками маршрутизации
- Реализовать ai/rate_limiter.py с отслеживанием дневных лимитов
- Подключить Groq (основной текстовый провайдер)
- Подключить OpenRouter (fallback)
- Реализовать обработчик /chat - основной чат с AI
- Добавить кэширование запросов (ai/cache.py)
- Проверить: /chat работает, fallback переключается, лимиты считаются

Этап 3: AI интеграция - изображения и мультимедиа (2-3 дня)

- Подключить Pollinations.ai для генерации изображений
- Реализовать /image - генерация картинок по описанию
- Подключить HuggingFace Whisper для транскрипции аудио
- Реализовать /transcribe - обработка голосовых сообщений
- Подключить HuggingFace NLLB для перевода

- Реализовать /translate - перевод текста
- Проверить: все мультимедиа-команды работают

Этап 4: Система платежей и монетизация (1-2 дня)

- Перенести систему Stars-платежей из stars-pay-bot
- Настроить три тарифа: Free / Pro / Ultra
- Реализовать проверку тарифа при каждом AI-запросе
- Настроить реферальную программу
- Реализовать /subscribe, /mylicenses, /refer
- Проверить: оплата Stars создаёт лицензию, лимиты зависят от тарифа

Этап 5: Mini App и публичный API (1-2 дня)

- Создать Mini App на GitHub Pages (miniapp/)
- Настроить deploy.yml для автоматического деплоя
- Подключить Mini App к боту через @BotFather
- Реализовать REST API для проверки лицензий (api/server.py)
- Настроить sync-data.yml для синхронизации licenses.json
- Проверить: Mini App открывается, оплата через него работает

Этап 6: Полировка и запуск (1-2 дня)

- Добавить оставшиеся AI провайдеры (GitHub Models, Gemini, Prodia)
- Реализовать /code для помощи с кодом
- Добавить inline-режим для текстовых запросов
- Настроить /admin и /providerstats
- Написать README.md с документацией
- Провести полное тестирование всех функций
- Запустить бота и начать привлечение пользователей

11. Зависимости проекта

Пакет	Версия	Назначение
aiogram	3.15+	Telegram Bot Framework (асинхронный)
aiosqlite	0.20+	Асинхронная работа с SQLite

httpx	0.27+	Асинхронные HTTP запросы к AI API
flask	3.1+	REST API сервер для проверки лицензий
groq	0.9+	Официальный SDK для Groq API
openai	1.30+	SDK для OpenRouter (совместимый API)
huggingface-hub	0.23+	HuggingFace Serverless Inference
google-generativeai	0.7+	Google Gemini API SDK
Pillow	10.0+	Обработка изображений
pyyaml	6.0+	Чтение YAML конфигураций

Таблица 12. Зависимости Python

12. Ключевые фрагменты кода

12.1 AI Router - маршрутизация запроса

```
class AIRouter:
    PROVIDER_CHAINS = {
        "text": ["groq", "openrouter", "github", "gemini", "huggingface"],
        "image": ["pollinations", "huggingface_img", "prodia"],
        "audio_stt": ["huggingface_whisper", "groq_whisper"],
        "audio_tts": ["huggingface_tts"],
        "translate": ["huggingface_nllb", "groq", "gemini"],
        "code": ["groq", "openrouter", "github"],
    }

    async def route(self, task_type: str, prompt: str,
                    user_id: int, **kwargs) -> AIResponse:
        tier = await self._get_user_tier(user_id)
        chain = self.PROVIDER_CHAINS.get(task_type, ["groq"])
        for provider_name in chain:
            provider = self.providers[provider_name]
            if not await self.limiter.can_use(
                provider_name, user_id, tier):
                continue
            try:
                result = await provider.generate(prompt, **kwargs)
                await self.limiter.record_usage(
                    provider_name, user_id, task_type)
            return result
```

```

except ProviderError:
    continue
raise AllProvidersExhaustedError(task_type)

```

12.2 Обработчик /chat

```

@router.message(F.text, ~F.text.startswith("/"))
async def handle_chat(message: Message):
    user_id = message.from_user.id
    tier = await db.get_user_tier(user_id)

    # Check daily limit
    usage = await db.get_daily_usage(user_id, "text")
    limit = TIER_LIMITS[tier]["text"]
    if usage >= limit:
        await message.answer(
            f"Лимит текстовых запросов исчерпан ({usage}/{limit}). "
            f"Подпишитесь на Pro для увеличения лимита: /subscribe")
        return

    # Route to AI
    await message.chat.do("typing")
    try:
        response = await ai_router.route(
            task_type="text",
            prompt=message.text,
            user_id=user_id,
            history=await db.get_chat_history(user_id)
        )
        await db.record_usage(user_id, "text", response.provider,
                               tokens=response.tokens_used)
        await message.answer(response.text)
    except AllProvidersExhaustedError:
        await message.answer(
            "Все AI провайдеры временно недоступны. "
            "Попробуйте через несколько минут.")

```

12.3 Платёж Stars - отправка инвойса

```

@router.callback_query(F.data.startswith("buy:"))
async def cb_buy(callback: CallbackQuery, bot: Bot):
    parts = callback.data.split(":")
    tier = parts[1] # "pro" or "ultra"
    period = parts[2] # "month", "year", "lifetime"
    plan = PLANS[tier][period]

```

```

# Check existing license

check = await db.check_user_license(callback.from_user.id)

if check.get("has_license"):

    await callback.answer("У вас уже есть подписка!", show_alert=True)

    return

await bot.send_invoice(

    chat_id=callback.from_user.id,

    title=f"AI Hub Bot - {plan['label']}",

    description=plan['description'],

    prices=[LabeledPrice(label=plan['label'], amount=plan['price'])],

    provider_token="", # Stars payments: empty

    payload=f"{tier}:{period}",

    currency="XTR", # Telegram Stars

    start_parameter=f"aih_{tier}_{period}",

)

await callback.answer()

```

13. Риски и способы их минимизации

Риск	Вероятность	Влияние	Митигация
Отмена бесплатных AI API	Средняя	Высокое	Мультипровайдерная архитектура с fallback цепочками; добавление новых провайдеров занимает 1-2 часа
GitHub Actions меняет политику бесплатных лимитов	Низкая	Критическое	Render.com как backup (render.yaml уже есть); Docker для портируемости
Превышение лимитов бесплатных API при росте аудитории	Средняя	Среднее	Многоуровневая система лимитов; кэширование; приоритизация платных пользователей
Telegram меняет политику Stars	Низкая	Среднее	Архитектура платежей изолирована от остального кода; замена платёжной системы - 1 день
Конкуренты с аналогичным продуктом	Высокая	Среднее	Первый запуск - ключевое преимущество; реферальная программа для вирусного роста; постоянное добавление новых AI

Таблица 13. Матрица рисков

Наиболее критический риск - это изменение политики GitHub Actions для публичных репозиторий. Однако даже в этом случае проект готов к миграции: Dockerfile, docker-compose.yml и render.yaml обеспечивают деплой на Render.com или любой другой PaaS за считанные минуты. Архитектура проекта не привязана к GitHub Actions аппаратно - это лишь один из способов хостинга, который можно заменить без изменения бизнес-логики.

14. Конкурентные преимущества проекта

Проект обладает рядом уникальных конкурентных преимуществ, которые отличают его от существующих решений на рынке. Во-первых, это нулевая стоимость эксплуатации: все AI API бесплатны, GitHub Actions для публичных репозиторий бесплатен, GitHub Pages бесплатен. Единственные расходы - это доменное имя (опционально) и время разработчика. Во-вторых, это масштабируемость: при необходимости можно добавить новые AI-провайдеры за 1-2 часа работы, просто создав новый адаптер в ai/providers/. В-третьих, это встроенная вирусность: реферальная программа мотивирует пользователей делиться ботом, а качественный AI-ответ в группе привлекает новых пользователей.

В отличие от sochiautoparts, чьи проекты (gitmoji-ai, devbadge, repokit) содержали фейковые Pro-функции и тривиально обходимые paywall, наш проект предоставляет реальные AI-сервисы с прозрачной системой лимитов. Пользователь получает ровно то, за что платит: увеличенные лимиты, доступ к быстрым моделям и расширенную историю чата. Это создаёт доверие и мотивирует к продлению подписки, в отличие от мошеннической модели, которая работает только до первого разоблачения.